Chapter 3_Evaluation Methodology

Evaluation aims to assess risks and uncover opportunities.
To mitigate risks, you first need to identify the places where your system is likely to fail and design your evaluation around them.

Challenges of evaluating base models

- The more intelligent the model becomes, the harder it is to evaluate them.
- The open-ended nature of foundation models undermines the traditional approach of evaluating a model against ground truth as there are so many possible correct response.
- models are treated as black-box, either because model providers choose not to expose models's details or because application developers lack the expertise to understand them.

Note: evaluation is not about assessing the model's performance on known tasks but also about discovering new pattern or work that AI can do.

Cross-entropy, perplexity, BPC, BPB are closely related. If you know the value of one, you can compute the other three, given the necessary information.

Recall: the model encodes the statistical information (How likely a token is to appear in a given context) about the languages. The more statistical information it can covers or captures, the better it is at predicting the next token.

Entropy

Entropy measures how much information, on average, a token carries. The higher the entropy, the more information each token carries, and the more bits are needed to represent a token.

In a recursive term, entropy measures how difficult it is to predict what comes next in a language.

- The lower a language's entropy --> the less information a token of a language carries and vice versa.

Cross entropy

Cross entropy tells us how difficult it is for the language model to predict what comes next in this dataset. = tells us how efficient a language model will be at compressing text.
It depends on two quality:

- The training data's predictability, measured by the training data's entropy.
- How the distribution captured by the language model diverges from the true distribution of the training data.

Perplexity

Perplexity is the exponential of entropy and cross entropy.
PPL measures the amount of uncertainty it has when predicting the next token.

- Higher uncertainty means there are more possible options for the next token.

Interpretation and use cases

- **More structured data gives lower expected perplexity** = more structured data is more predictable
- **The bigger the vocabulary, the higher the perplexity** = the more possible tokens there are, the harder it is for the model to predict the next token
- **The longer the context length, the lower the perplexity** = the more context a model has, the less uncertainty it will have in predicting the next token.

In AI engineering flow, perplexity is a good proxy for a model's capabilities. If a model's bad at predicting the next token --> its performance on down-stream tasks will also likely be bad.

- Perplexity is the lowest for texts that the model memorized during training
  Perplexity can be used to detect whether a text was in a model's training data. This is useful for detecting data contamination.
- Perplexity is the highest for unpredictable texts, such as texts expressing unusual ideas (like "my dog teaches quantum physics in his free time.") or gibberish ("home cat go eye"). Therefore, perplexity can be used to detect abnormal text.

Exact evaluation

- **Exact evaluation** produces judgement without ambiguity.
  Example: If the answer to a multiple-choice question is A and you pick B, your answer is wrong.
- **Subjective evaluation** can have ambiguity
  Example: an essay's score depends on who grades the essay.

Two approaches that produce exact evaluation: for open-ended response (arbitrary text generation)

- **Functional correctness**: evaluating a system based on whether it performs the intended functionality.
  - **For example:** if you ask a model to create a website, does the generated website meet your requirements? If you ask a model to make a reservation at a certain restaurant, does the model succeed?
    Functional correctness = measures whether your application does what it is intended to do.

- **Similarity measurements against reference data:** evaluate AI's outputs against reference data.
  - **For example:** if you ask a model to translate a sentence from French to English, you can evaluate the generated English translation against the correct English translation.
  - **Reference response** = ground truths or canonical response
    - Reference response can be generated typically by humans and increasingly by AIs
  - Metrics that require references are **reference-based**, and metrics that don't are **reference-free.**
  - There are fours ways to measure the similarity between two open-ended texts:
    - Asking an evaluator to make the judgement whether two texts are the same
    - **Exact match:** whether the generated response matches one of the reference responses exactly
    - **Lexical similarity:** how similar the generated response looks to the reference responses
    - **Semantic similarity:** how close the generated response is to the references in meaning (semantics)
  - And, two responses can be compared by human evaluators or AI evaluators.
  - We can also use the similarity measurement in cases including:
    - **Retrieval and search:** find items similar to a query
    - **Ranking:** rank items based on how similar they are to a query
    - **Clustering:** Cluster items based on how similar they are to each other
    - **Anomaly detection:** detect items that are the least similar to the rest
    - **Data deduplication:** remove items that are too similar to other items
  - **Lexical similarity:** measures how much two texts overlap. You can do this by first breaking each text into smaller tokens. In its simplest form, lexical similarity can be measured by counting how many tokens two texts have in common.
    - **Example:**
      - My cats eat the mice
      - Cats and mice fight all the time
    - One way to measure lexical similarity is *approximate string matching*, known *colloquially* as *fuzzy matching.* It measures the similarity between two texts by counting how many edits it's need to convert from one text to another, a number called *edit distance.*
      - **Example:**
        - Deletion: "brad" --> "bad"
        - Insertion: "bad" --> "bard"
        - Substitution: "bad" --> "bed"
    - Another way to measure it, *n-gram similarity*, measured based on the overlapping of sequences of tokens, n-gram, instead of single token.

    - **Common metric:** BLEU, ROUGE, METEOR++, TER, and CIDEr. They differ in how the overlapping is calculated.
    - **Limitation**
        - it requires curating a comprehensive set of reference responses. **Example**: a good response can get a low similarity score if the reference set does not contain any response that look like it
        - Low-quality reference data is one of the reasons that reference-free metrics were strong contenders for reference-based metrics in terms of correlation to human judgement.
        - Higher lexical similarity scores don't always mean better responses. Example: on Human-eval, a code generation benchmark, OpenAI found that BLEU scores for incorrect and correct solutions were similar.
- **Semantic similarity:** aims to compute the similarity in semantics. This first requires transforming a text into a numerical representation, which is called an **embedding**
    - The similarity between two embeddings can be computed using metrics such as cosine similarity. Two embedding that are exactly the same have a similarity score of 1. Two opposite embeddings have similarity score of 0.
    - Semantic similarity can be computed for embeddings of any data modality (any form, format... like image, text, video...)
    - **Cosine similarity:**

$$cos(\theta) = \frac{A \cdot B}{||A|| \cdot ||B||}$$

        - $||A|| = \sqrt{A_1^2 + A_2^2 + \dots}$
        - $||B|| = \sqrt{B_1^2 + B_2^2 + \dots}$
    - Semantic similarity does not need a set of reference responses as comprehensive as lexical similarity does.
    - Semantic similarity depends on the quality of the underlying embedding algorithm.
        - **For example:** Two texts with the same meaning can still have a low semantic similarity score if their embeddings are bad.

Embedding 101

An embedding is a numerical representation that aims to capture the meaning of the original data.

An embedding is a vector.

For example: "The cat sits on a mat." --> embedding vector [0.11, 0.02, 0.54]

In reality, the size of an embedding vector (the number of elements in the embedding vector) is typically between 100 and 10000.

Why we need to embed?
Simply because models typically their inputs to first be transformed into vector representation, many ML models, including GPTs and Llamas, also involves a step to generative embeddings.

- The goal of embedding algorithm is to produce embeddings that capture the essence of the original data.
- An embedding algorithm is considered good if more-similar texts have closer embeddings, measured by Cosine similarity or related metrics.
- Embeddings are used in tasks including classification, topic modeling, recommender systems, and RAG.

Joint Embedding
A joint embedding space that can represent data of different modalities is a *multimodal embedding space.*
For example: in a text-image joint embedding space, the embedding of an image of a man fishing should be closer to the embedding of the text "a fisherman" than the embedding of the text "fashion show" ,

AI as a judge

AI not only can evaluate the result, it can also interpret its decision. --> Useful when auditing the result.

Use AI as a judge

1. Evaluate the quality of the response by itself, given the original question.
2. Compare a generated response to a reference response
3. Compare two generated responses and determine which one is better or predict which one will likely prefer.

Prompt:

- The task the model is to perform, such as to evaluate the relevance between a generated answer and the question.
- The criteria the model should follow to evaluate, such as "Your primary task should be on determining whether the generated answeer contains sufficient information to address the given question according to the ground truth answer" The more detail the instruction the better.
- The scoring system:
  - Classification: good/bad, relevant/irrelevant/neutral
  - Discrete numerical value, such as 1-5 (Can also be considered as classification)
  - Continuous numerical values, such as betweeen 1 and 0 e.g., when you want to measure the degree of similarity.

Limitations of AI as a judge

- Latency cost
- Nontrivial cost
- Inconsistency
- Criteria ambiguity: AI as a judge is not standardized, making it easy to misinterpret and misuse them.
    - MLFlow, Ragas, and LlamaIndex all have the built-in criterion faithfulness to meeasure how faithful A generated output is to the given context, but the scoring systems are all different.
- Biases: An AI may favor the first answer in a pairwise comparison or the first in a list of options. This can be mitigated by representing the same multiple times with different orderings or with carefully crafted prompt.
    - The position bias of AI is the opposite of that of humans. Humans tend to favor the answer they see last, which is called recency bias.

Specialized AI as as judge

- Reward model
- Reference-based judge: evaluated the generated response with respect to one or more references
- Preference model: which of the two response is better